

work has now become a part of the much broader SciPy project. You can find out more about it at www.scipy.org/History_of_SciPy.

Python does not come with the NumPy module included by default. It is, however, available in various distributions. You will need to make sure that it is installed. Familiarity with core Python is assumed.

NumPy one-dimensional arrays or vectors

The best way to learn is to try a little code. Suppose you need to compute corresponding elements of two arrays. How would the code and performance be different? Try the following small benchmark:

```
N=1000000
a=[i for i in range(N)]
b=[0.5*i for i in range(N)]
for i in range(10):
    ab=[a[i]*b[i] for i in range(N)]
```

You have defined two arrays of a million elements each. Your code creates a new array, which is the product of the corresponding elements. You repeat it 10 times so that the execution times can be compared reasonably.

Now, you can write the same code using NumPy arrays.

```
import numpy as np
N=1000000
a=np.array([i for i in range(N)])
b=np.array([0.5*i for i in range(N)])
for i in range(10):
    ab=a*b
```

The method `array` in the NumPy module converts a Python list into an array object of uniform type elements. There are a number of operations defined on these objects—for example, multiplication results in a new array, which is the product of the corresponding elements.

On my system, the core Python program took 8.9 seconds while the NumPy version took 1.7 seconds.

Now, try the following code:

```
>>> a=[1,2,3]
>>> b=[3,2,1]
>>> a+b
```

Before you give your answer, try the following as well:

```
>>> s=['a','c']
>>> a+s
```

The only reasonable, consistent answer is concatenation of the two lists. Now, try the same thing with NumPy arrays:

```
>>> a=np.array([1,2,3])
>>> b=np.array([3,2,1])
```

```
>>> s=np.array(['a','c'])
>>> a+b
array([4, 4, 4])
>>> a+s
```

Since the elements in the array are of the same type, summation of `a` and `b` is very reasonable. Now, what should happen in the case of adding `s` to `a`? Understandably, it should raise an exception, as the addition of a number to a string is not meaningful.

You need to compute the sine of a list of numbers. This can also be a good test to benchmark. So, try the following code:

```
import math
N=1000000
a=[i**2*math.pi/N for i in range(N)]
for i in range(10):
    sina=[math.sin(v) for v in a]
```

You have defined an array of one million equally spaced values between 0 and 2π . As before, create a new array containing the sine of the corresponding value in the first array. Repeat it 10 times so that the run time is large enough.

The corresponding program using NumPy will be:

```
import math
import numpy as np
N=1000000
a=np.array([i**2*math.pi/N for i in range(N)])
for i in range(10):
    sina=np.sin(a)
```

As before, you have converted a Python list into a NumPy array. You can pass an array to a function and its meaning is very clear. It creates a new array whose values are the function of each of the corresponding elements of the array, passed as a parameter.

In this case, on my system, the core Python version takes 7.3 seconds while the NumPy version takes 1.8 seconds.

NumPy multi-dimensional arrays and matrices

A list of lists would be a two dimensional array. Try another little test:

```
r=1280*11
m=1024*v
for i in range(10):
    m2=[]
    for i in m:
        m2.append(3.5*v for e in r)
```

Define a list of 1,024 rows. Each row has 1,280 elements, with a value of 1. The task is to scale all